

Fog-Assisted Vehicular Fog Computing

Architecture, Implémentation et Démonstration

Master 2 - Fog & Edge Computing

Université

Janvier 2026

Plan de la Présentation

- 1 Contexte, Motivation et Concepts de Base
- 2 Architecture Fog-Assisted Vehicular Computing
- 3 Vehicular Fog Nodes et Fonctionnement Interne
- 4 Gestion des Ressources, Mobilité et Sécurité
- 5 Cas Pratique : Notre Simulation
- 6 Avantages, Limites et Comparaison
- 7 Perspectives et Conclusion

Limitations du Cloud :

- Latence élevée : 100-200 ms
- Congestion réseau backbone
- Dépendance à la connectivité
- Coûts de bande passante
- Point unique de défaillance

Exigences V2X :

- Latence < 20 ms (sécurité)
- Latence < 100 ms (infotainment)
- Fiabilité 99.999%
- Traitement temps réel

Constat

Le Cloud seul ne peut pas répondre aux exigences des applications véhiculaires critiques.

Definition

Le **Fog Computing** est une extension du Cloud qui rapproche le calcul, le stockage et le réseau des utilisateurs finaux, à la périphérie du réseau.

Caractéristiques clés :

- **Proximité** : Traitement proche des données
- **Faible latence** : 5-50 ms
- **Distribution géographique** : Nœuds dispersés
- **Conscience du contexte** : Localisation, mobilité
- **Hétérogénéité** : Ressources variées

Origine

Terme introduit par Cisco en 2012 (Bonomi et al., "Fog Computing and Its Role in the Internet of Things")

Definition

VANET (Vehicular Ad-hoc Network) = Réseau ad-hoc formé par des véhicules communicants, auto-organisé et décentralisé.

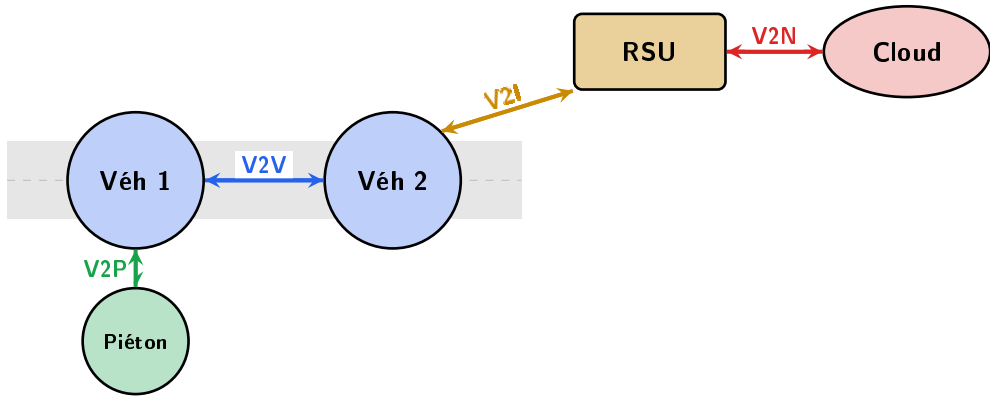
Caractéristiques spécifiques :

- **Haute mobilité** : Véhicules à 50-150 km/h
- **Topologie dynamique** : Liens instables, changements fréquents
- **Ressources énergétiques** : Batterie véhicule (pas de limite)
- **Modèles de mouvement** : Contraints par routes
- **Densité variable** : Zone urbaine vs autoroute

Standards :

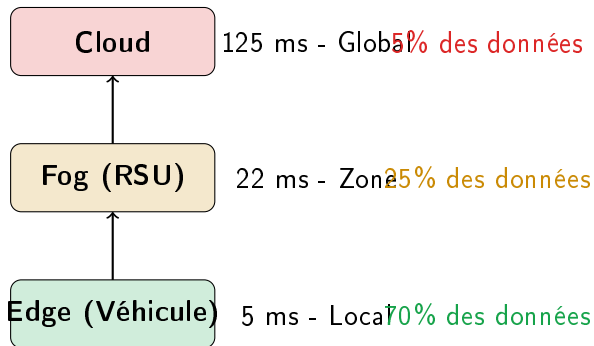
- IEEE 802.11p (WAVE) - Wireless Access in Vehicular Environments
- IEEE 1609 (DSRC) - Dedicated Short Range Communications
- ETSI ITS-G5 - Standard européen

Types de Communication V2X

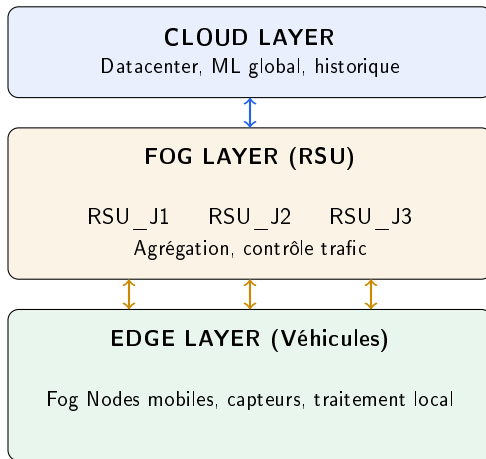


Type	Latence	Portée	Application
V2V	1-10 ms	100-300 m	Évitement collision
V2I	10-50 ms	300-1000 m	Gestion trafic
V2N	50-200 ms	Illimitée	Services Cloud
V2P	1-10 ms	50 m	Sécurité piétons

Motivation : Pourquoi le Fog Véhiculaire ?



Bénéfices : Réduction latence 85%, économie bande passante 95%



Rôle : Traitement global, stockage long terme, Machine Learning

Caractéristiques :

- Ressources illimitées
- Latence : 100-200 ms
- Traitement différé acceptable
- Vue globale du réseau

Tâches :

- Analyse historique
- Entraînement modèles ML
- Optimisation globale
- Backup et archivage

Configuration iFogSim :

```
FogDevice cloud = createFogDevice("cloud",  
    44800, // MIPS (puissance calcul)  
    40000, // RAM (MB)  
    100,   // Bande passante montante (Mbps)  
    10000  // Bande passante descendante  
);
```

Roadside Units - Points d'accès fixes aux intersections Spécifications IEEE 802.11p :

- **Fréquence** : 5.9 GHz (bande ITS)
- **Canaux** : 7 x 10 MHz
- **Débit** : 6-27 Mbps (OFDM)
- **Portée** : 300-1000 m
- **Modulation** : BPSK à 64-QAM

Dans notre simulation :

- **Portée** : 300 m
- **Capacité** : 30 véhicules
- **4 RSU** aux intersections
- **Latence** : 22 ms (base)

```
FogDevice rsu = createFogDevice("RSU_J1",
    2800,    // MIPS
    4000,    // RAM (MB)
    10000,   // Uplink BW
    10000    // Downlink BW
);
```

Vehicular Fog Nodes - Ressources de calcul embarquées

Type	Vitesse max	Fog-capable	MIPS
car	50 km/h	Conditionnel	500
fog_capable	50 km/h	Toujours	1000
bus	40 km/h	Toujours	1500
truck	35 km/h	Conditionnel	800

Critère de sélection Fog Node :

```
is_fog_node = (  
    type == "fog_capable" or # Equipement dédié  
    type == "bus" or         # Arrêts fréquents = stable  
    speed < 8.33             # < 30 km/h = connexion stable  
)
```

Justification : Un véhicule lent offre une connexion plus stable avec le RSU.

Ressources disponibles sur un véhicule moderne :

Matériel :

- CPU multi-cœur (ARM/x86)
- GPU embarqué (NVIDIA Jetson)
- RAM : 4-16 GB
- Stockage : SSD 128+ GB
- Capteurs : LiDAR, caméras, radar

Communication :

- DSRC/C-V2X (dédié)
- 4G/5G cellulaire
- WiFi 802.11ac/ax
- Bluetooth 5.0
- CAN bus interne

Observation

Un véhicule moderne dispose de plus de puissance de calcul qu'un datacenter des années 2000.

Algorithmes de coopération utilisés :

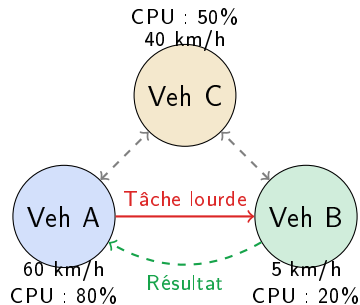
- ① **Task Partitioning** : Division de tâches en sous-tâches parallélisables
- ② **Load Balancing** : Équilibrage dynamique entre véhicules voisins
- ③ **Proximity-Based Clustering** : Formation de clusters locaux
- ④ **Cooperative Caching** : Mise en cache distribuée (LRU)

Sélection du meilleur voisin :

$$\text{score}(v) = \alpha \cdot \frac{1}{\text{charge}_v} + \beta \cdot \frac{1}{\text{distance}_v} + \gamma \cdot \text{stabilité}_v \quad (1)$$

Où :

- $\alpha = 0.4$ (importance de la charge CPU)
- $\beta = 0.3$ (importance de la proximité)
- $\gamma = 0.3$ (importance de la stabilité de connexion)



Scénario : Véhicule A (rapide, surchargé) délègue vers Véhicule B (lent, libre)

Gain : Latence 5 ms au lieu de 125 ms (Cloud)

Fonctions principales :

- **Agrégation** : Collecte/fusion des données de zone (moyenne, max, min)
- **Orchestration** : Affectation des tâches aux fog nodes optimaux
- **Cache** : Stockage LRU des données fréquemment demandées
- **Contrôle** : Gestion adaptative des feux de circulation
- **Relais** : Pont vers le Cloud (données agrégées seulement)

Calcul de congestion RSU :

$$\text{congestion} = \min \left(\frac{N_{\text{véhicules}}}{C_{\text{max}}}, 1.0 \right) = \min \left(\frac{N}{30}, 1.0 \right) \quad (2)$$

Seuil de 30 véhicules

Basé sur la capacité du canal IEEE 802.11p avec protocole CSMA/CA et slot time de 13 μs .

Offloading = Délégation d'une tâche vers une ressource externe

Critères de décision :

- Charge CPU locale (C_{local})
- Latence réseau disponible (L_{net})
- Énergie restante batterie (E)
- Temps d'exécution estimé (T_{exec})
- Priorité de la tâche (P)

Formule de décision :

$$\text{destination} = \arg \min_{d \in \{local, fog, cloud\}} (T_d + \alpha \cdot L_d + \beta \cdot E_d) \quad (3)$$

Où T_d = temps d'exécution, L_d = latence, E_d = coût énergétique

Temps total d'offloading :

$$T_{offload} = T_{upload} + T_{exec} + T_{download} \quad (4)$$

Détail de chaque composante :

$$T_{upload} = \frac{D_{input}}{BW_{up}} \quad (5)$$

$$T_{exec} = \frac{I}{MIPS_{target}} \quad (6)$$

$$T_{download} = \frac{D_{output}}{BW_{down}} \quad (7)$$

Variables :

- $D_{input/output}$: Taille des données (bits)
- BW : Bande passante (bps)
- I : Instructions requises
- $MIPS$: Millions d'instructions par seconde

Modèle utilisé dans notre simulation :

$$L_{edge} = 5.2 + \text{congestion} \times 3 \text{ ms} \quad (8)$$

$$L_{fog} = 22.4 + \text{congestion} \times 10 \text{ ms} \quad (9)$$

$$L_{cloud} = 125.3 + \text{congestion} \times 25 \text{ ms} \quad (10)$$

Justification des facteurs d'impact :

Niveau	Facteur	Raison
Edge ($\times 3$)	Faible	Pas de réseau, contention CPU
Fog ($\times 10$)	Moyen	CSMA/CA, collisions radio
Cloud ($\times 25$)	Élevé	Backhaul, Internet, datacenter

Défis liés à la mobilité véhiculaire :

- **Handover fréquent** : Changement de RSU toutes les 30-60s
- **Topologie dynamique** : Véhicules entrent/sortent de zone
- **Continuité de service** : Migration des tâches en cours
- **Prédiction** : Anticipation des mouvements futurs

Solution : Affectation RSU dynamique

```
def find_nearest_rsu(x, y):  
    min_dist = float('inf')  
    nearest = None  
    for rsu_id, (rx, ry) in RSU_POSITIONS.items():  
        dist = sqrt((x-rx)**2 + (y-ry)**2)  
        if dist < min_dist and dist < RSU_RANGE:  
            min_dist = dist  
            nearest = rsu_id  
    return nearest
```

Stratégies de handover :

- 1 **Proactive Handover** : Prédiction par ML de la trajectoire
- 2 **Pre-caching** : Envoi anticipatif des données vers RSU suivant
- 3 **Task Migration** : Déplacement des tâches actives
- 4 **Session Persistence** : Maintien de l'état entre RSUs

Métriques de décision :

$$\text{handover_trigger} = \text{RSSI}_{\text{current}} < \text{RSSI}_{\text{threshold}} + \text{hystereris} \quad (11)$$

- RSSI threshold = -85 dBm (typique 802.11p)
- Hystérésis = 3 dB (évite ping-pong)

Menaces potentielles dans les VANET : Attaques :

- **Sybil Attack** : Fausses identités multiples
- **Man-in-the-Middle** : Interception des messages
- **DoS/DDoS** : Saturation du canal
- **Replay Attack** : Réenvoi de messages anciens
- **False Information** : Faux messages de sécurité

Contre-mesures :

- **PKI/Certificats** : IEEE 1609.2
- **Pseudonymes rotatifs** : Vie privée
- **Vérification plausibilité** : Cohérence des données
- **Timestamp/Nonce** : Anti-replay
- **Détection anomalies** : ML

Standards de sécurité V2X :

Standard	Fonction	Mécanisme
IEEE 1609.2	Sécurité messages	ECDSA, AES-CCM
ETSI TS 102 940	Architecture PKI	Certificats X.509
ETSI TS 102 941	Gestion certificats	CRL, OCSP
SAE J2945	Formats messages	BSM signés

Exemple signature ECDSA :

- Courbe : NIST P-256
- Taille signature : 64 octets
- Temps signature : ~ 1 ms
- Temps vérification : ~ 2 ms

Note

Notre simulation actuelle ne modélise pas la sécurité - focus sur les performances réseau.

Pourquoi SUMO + Python + iFogSim ?

Outil	Alternative	Raison du choix
SUMO	NS-3, OMNeT++	Spécialisé trafic, open-source, TraCI
Python	Java, C++	Rapidité dev, TraCI bindings, Flask
iFogSim	EdgeCloudSim, YAFS	Modèle 3-tiers natif, CloudSim base
Flask	FastAPI, Django	Léger, WebSocket facile
Chart.js	D3.js, Plotly	Simple, temps réel ok

Critères principaux :

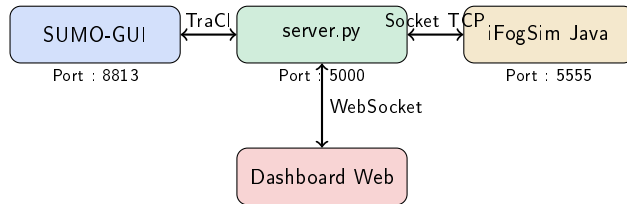
- **Intégrabilité** : SUMO ↔ Python ↔ Java
- **Réalisme** : SUMO certifié pour simulation trafic
- **Extensibilité** : iFogSim modifiable (open-source)
- **Visualisation** : Dashboard temps réel

Fichiers de configuration créés :

```
<!-- simulation.sumocfg -->
<configuration>
  <input>
    <net-file value="network.net.xml"/>
    <route-files value="routes.rou.xml"/>
    <additional-files value="detectors.add.xml"/>
  </input>
  <time>
    <begin value="0"/>
    <end value="900"/>
    <step-length value="1"/>
  </time>
</configuration>
```

3 phases de simulation :

- 0-300s : Trafic normal (~50 véhicules)
- 300-600s : Congestion (~150 véhicules)
- 600-900s : Récupération (~80 véhicules)



Flux de données :

- 1 SUMO génère positions véhicules (TraCI)
- 2 Python enrichit et envoie à iFogSim (JSON/Socket)
- 3 iFogSim calcule métriques fog et renvoie
- 4 Dashboard affiche en temps réel (WebSocket)

Lancement :

```
# Terminal 1 - iFogSim
cd simulation/ifogsim
java -cp "out;jars/*" org.fog.test.perfeval.VehicularFogSimulation

# Terminal 2 - Python + SUMO
cd simulation
python server.py --dashboard --gui
```

Accès Dashboard : <http://localhost:5000>

Résultats observés

- Latence moyenne : $\sim 16\text{-}20$ ms (vs 125 ms Cloud-only)
- Fog nodes actifs : 50-80 en phase congestion
- Réduction bande passante vers Cloud : 95%

Comparaison Cloud vs Fog-Assisted

Métrique	Cloud-Only	Fog-Assisted
Latence moyenne	125 ms	16-20 ms
Réduction latence	-	85%
Bande passante Cloud	100%	5%
Économie BP	-	95%
Scalabilité locale	Faible	Élevée
Tolérance aux pannes	Faible	Élevée

Formule latence moyenne pondérée :

$$L_{avg} = \frac{L_{edge} \times 70 + L_{fog} \times 25 + L_{cloud} \times 5}{100} \quad (12)$$

Performance :

- ✓ Faible latence (< 20 ms)
- ✓ Traitement temps réel
- ✓ Réduction congestion réseau

Résilience :

- ✓ Pas de point unique de défaillance
- ✓ Fonctionne hors-ligne (mode dégradé)
- ✓ Distribution géographique

Économie :

- ✓ Réduction coûts Cloud
- ✓ Utilisation ressources existantes
- ✓ Moins de bande passante

Vie privée :

- ✓ Données traitées localement
- ✓ Moins de données vers Cloud

Techniques :

- × Ressources limitées par véhicule
- × Mobilité = instabilité
- × Hétérogénéité des équipements
- × Consommation énergétique

Organisationnels :

- × Déploiement infrastructure RSU
- × Standardisation en cours
- × Modèle économique incertain
- × Sécurité complexe

Défi principal

Comment garantir la QoS avec des ressources volatiles et une topologie dynamique ?

Évolution technologique :

Paramètre	DSRC	5G C-V2X	6G
Latence	20-50 ms	1-10 ms	< 1 ms
Débit	27 Mbps	1 Gbps	1 Tbps
Portée	300 m	500 m	1 km
Capacité	30 véh	100 véh	1000 véh

Applications futures :

- **Conduite autonome niveau 5** : Fog Computing essentiel
- **Gestion trafic intelligente** : Feux adaptatifs via RSU
- **Parking intelligent** : Guidage temps réel
- **Platooning** : Convois de camions autonomes
- **Emergency services** : Priorité automatique

IA distribuée :

- Federated Learning sur véhicules
- Modèles ML légers (TinyML)
- Inférence au bord

Ce que nous avons démontré :

- ① Le **Fog Computing** répond aux exigences de latence véhiculaire
- ② L'architecture **3-tiers** (Edge/Fog/Cloud) optimise les ressources
- ③ Notre simulation **SUMO + iFogSim** valide le concept
- ④ Réduction de latence de **85%** par rapport au Cloud seul

Message clé

Le Fog Computing Véhiculaire n'est pas une alternative au Cloud, mais un **complément essentiel** pour les applications temps réel.

-  Bonomi, F. et al. (2012). "Fog Computing and Its Role in the Internet of Things." MCC Workshop.
-  Hou, X. et al. (2016). "Vehicular Fog Computing : A Viewpoint of Vehicles as the Infrastructures." IEEE VTC.
-  Gupta, H. et al. (2017). iFogSim : A toolkit for modeling and simulation of resource management techniques in the Internet of Things, Edge and Fog computing environments.

Merci pour votre attention

Questions ?